

الدليل العملي للمحاضرة الأولى

هندسة المتطلبات، إدارة العمل الجماعي، وتأسيس بيئة Git

المقرر: هندسة البرمجيات - الجانب العملي

العام الجامعي: 2026/2027

مقدم المقرر: المهندس ساهر الهمداني

مدة المحاضرة: 90 دقيقة

نوع النشاط: عمل جماعي تطبيقي

الأدوات المستخدمة: Git, GitHub, GitHub Issues, GitHub Projects, Markdown

1. وصف المحاضرة

تمثل هذه المحاضرة نقطة الانتقال من العمل البرمجي الفردي غير المنظم إلى العمل الهندسي الجماعي القابل للإدارة والتوسع.

لن يبدأ الطلاب بكتابة الكود مباشرة، بل سيبدؤون بالخطوات التي تسبق كتابة الكود في المشاريع الحقيقية:

1. فهم المشكلة.
2. تحديد المستخدمين.
3. توثيق المتطلبات.
4. تحويل المتطلبات إلى مهام قابلة للتنفيذ.
5. توزيع العمل على أعضاء الفريق.
6. استخدام Git وGitHub لإدارة التغييرات.
7. مراجعة العمل قبل دمجها في المشروع.

الهدف ليس إنشاء مشروع كامل خلال المحاضرة، بل تأسيس طريقة عمل احترافية سيستمر الفريق باستخدامها طوال الفصل الدراسي.

2. نواتج التعلم

بنهاية هذه المحاضرة، يُتوقع من الطالب أن يكون قادراً على:

- التمييز بين المبرمج ومهندس البرمجيات.
- تحديد المتطلبات الوظيفية وغير الوظيفية للنظام.
- إعداد وثيقة متطلبات برمجية مصغرة Mini-SRS.
- كتابة قصص مستخدم واضحة وقابلة للتنفيذ.
- صياغة معايير قبول قابلة للاختبار.
- اكتشاف حالات الحافة والمواقف غير الطبيعية.
- التمييز بين قائمة الأعمال Backlog ولوحة Kanban.
- إنشاء مستودع GitHub منظم.
- إنشاء Issues وربطها بمتطلبات المشروع.
- إنشاء Branch مستقل لتنفيذ مهمة.

- تنفيذ عمليات Push و Commit بطريقة صحيحة.
- فتح Pull Request ومراجعة التغييرات قبل دمجها.
- توثيق استخدام أدوات الذكاء الاصطناعي بشفافية.

3. الخطة الزمنية للمحاضرة

الزمن	النشاط
0-5 دقائق	المقدمة والتحفيز
5-25 دقيقة	هندسة المتطلبات وMini-SRS
25-40 دقيقة	User Stories و Acceptance Criteria و Edge Cases
40-50 دقيقة	Backlog و Kanban Board
50-60 دقيقة	شرح دورة Git و GitHub
60-85 دقيقة	التطبيق العملي الجماعي
85-90 دقيقة	مراجعة المخرجات والتقييم

4. المقدمة والتحفيز

من المبرمج إلى مهندس البرمجيات

يمكن تقديم النص التالي للطلاب في بداية المحاضرة:

يستطيع شخص واحد بناء خيمة خلال ساعات قليلة، باستخدام أدوات بسيطة ودون الحاجة إلى مخططات هندسية مفصلة. وإذا حدث خطأ أثناء البناء، يمكن فك الخيمة وإعادة تركيبها بتكلفة محدودة.

لكن هل يمكن استخدام الأسلوب نفسه لبناء ناطحة سحاب؟

لا يمكن البدء بوضع الحديد وصب الخرسانة قبل دراسة الأرض. وتحديد المتطلبات، وإعداد المخططات، وتوزيع المسؤوليات، ووضع معايير السلامة، ومراجعة كل مرحلة قبل الانتقال إلى المرحلة التالية.

البرامج الصغيرة تشبه الخيمة. قد يتمكن مبرمج واحد من بنائها بطريقة مباشرة، حتى لو لم تكن المتطلبات مكتوبة أو الكود منظماً.

أما الأنظمة التي تخدم آلاف أو مئات الآلاف من المستخدمين فهي أقرب إلى ناطحات السحاب. لا يكفي فيها أن يعمل الكود على جهاز المبرمج، بل يجب أن يكون النظام:

- صحيحاً.
- آمناً.
- قابلاً للصيانة.

- قابلاً للتوسع.
- مفهوماً لبقية أعضاء الفريق.
- قابلاً للاختبار والمراجعة.

المبرمج يسأل: كيف أكتب هذا الكود؟

أما مهندس البرمجيات فيسأل:

- لماذا نبني هذه الخاصية؟
- من سيستخدمها؟
- ما المشكلة التي تحلها؟
- ما الحالات التي قد تفشل فيها؟
- كيف سنختبرها؟
- كيف سيعمل عليها أكثر من مطور دون أن يفسد أحدهم عمل الآخر؟
- كيف يمكن تطويرها بعد سنة أو خمس سنوات؟

خلال هذا المقرر لن نتعلم فقط كيف نكتب الكود، بل كيف نبني أنظمة برمجية بطريقة هندسية تشبه أساليب العمل في الفرق التقنية الاحترافية.

5. هندسة المتطلبات

5.1 ما المقصود بالمتطلب؟

المتطلب هو وصف واضح لشيء يحتاجه المستخدم أو يجب أن يحققه النظام.

قد يصف المتطلب:

- وظيفة ينفذها النظام.
- سلوكاً متوقعاً.
- مستوى أداء.
- قيداً أمنياً.
- قاعدة عمل.
- نوعاً من المستخدمين.
- طريقة تعامل النظام مع الأخطاء.

هندسة المتطلبات لا تعني كتابة قائمة أمنيات، بل تعني اكتشاف احتياجات أصحاب المصلحة وتحليلها وتوثيقها وتحويلها إلى عناصر قابلة للتنفيذ والاختبار.

5.2 لماذا نفشل عندما تكون المتطلبات غير واضحة؟

عندما يبدأ الفريق بالبرمجة قبل فهم المتطلبات، تظهر مشكلات مثل:

- بناء خاصية لا يحتاجها المستخدم.
- اختلاف فهم أعضاء الفريق للمطلوب.

- إعادة كتابة أجزاء كبيرة من النظام.
- صعوبة تحديد هل المهمة انتهت أم لا.
- ظهور حالات لم يتم التفكير فيها.
- تأخر المشروع وتضخم تكلفته.
- خلافات بين العميل وفريق التطوير.

المشكلة ليست دائماً في قدرة المبرمج على كتابة الكود، بل قد تكون في أن الفريق كتب الكود الصحيح لمشكلة خاطئة.

6. المتطلبات الوظيفية وغير الوظيفية

6.1 المتطلبات الوظيفية Functional Requirements

تصف المتطلبات الوظيفية ما الذي يجب أن يفعله النظام.

أمثلة

- يجب أن يتمكن الطالب من إنشاء حساب.
- يجب أن يتمكن المستخدم من تسجيل الدخول.
- يجب أن يتمكن المشرف من إضافة مقرر جديد.
- يجب أن يتمكن الطالب من رفع واجب.
- يجب أن يرسل النظام إشعاراً عند اقتراب موعد التسليم.
- يجب أن يتمكن المدير من استخراج تقرير بالطلاب المسجلين.

السؤال الذي تجيب عنه المتطلبات الوظيفية هو:

ما الوظائف التي يقدمها النظام؟

6.2 المتطلبات غير الوظيفية Non-Functional Requirements

تصف المتطلبات غير الوظيفية جودة النظام أو القيود التي يجب أن يعمل ضمنها.

أمثلة

- يجب ألا تستغرق صفحة تسجيل الدخول أكثر من ثانيتين في الظروف الطبيعية.
- يجب تخزين كلمات المرور بصورة مشفرة.
- يجب أن يعمل النظام على الهواتف وأجهزة الحاسوب.
- يجب أن يكون النظام قادراً على خدمة 5,000 مستخدم متزامن.
- يجب تسجيل المحاولات الفاشلة لتسجيل الدخول.
- يجب أخذ نسخة احتياطية من البيانات بصورة دورية.
- يجب أن تكون واجهة المستخدم متاحة باللغتين العربية والإنجليزية.

السؤال الذي تجيب عنه المتطلبات غير الوظيفية هو:

كيف يجب أن يعمل النظام، وما مستوى الجودة المطلوب؟

6.3 مقارنة مختصرة

الجانِب	متطلب وظيفي	متطلب غير وظيفي
التركيز	وظيفة النظام	جودة النظام أو قيوده
المثال	يستطيع المستخدم تسجيل الدخول	تتم عملية تسجيل الدخول خلال ثانيتين
الاختبار	هل نفذ النظام الوظيفة؟	هل نفذها بالمستوى المطلوب؟
التأثير	يحدد ما الذي سنبنيه	يحدد مستوى جودة ما سنبنيه

6.4 مثال تطبيقي

لنفترض أننا نريد إنشاء نظام لإدارة مهام الطلاب.

متطلبات وظيفية

- يستطيع الطالب إنشاء مهمة.
- يستطيع الطالب تعديل المهمة.
- يستطيع الطالب حذف المهمة.
- يستطيع الطالب تحديد موعد نهائي للمهمة.
- يستطيع الطالب تعليم المهمة كمكتملة.

متطلبات غير وظيفية

- يجب ألا تستغرق عملية عرض المهام أكثر من ثانيتين.
- يجب ألا يستطيع أي طالب رؤية مهام طالب آخر.
- يجب أن يعمل النظام على شاشة الهاتف.
- يجب حفظ سجل بالأخطاء المهمة.
- يجب التحقق من صحة البيانات قبل تخزينها.

7. وثيقة SRS المصغرة

7.1 ما المقصود بـ SRS؟

SRS هي اختصار لعبارة:

Software Requirements Specification

أي: وثيقة مواصفات متطلبات البرمجيات.

تصف الوثيقة النظام المطلوب بناؤه، والمستخدمين المستهدفين، والوظائف الأساسية، والقيود، ومعايير الجودة.

في المشاريع الكبيرة قد تكون وثيقة SRS طويلة ومفصلة. أما في هذه المحاضرة فسيستخدم الطلاب نسخة مصغرة تركز على المعلومات الأساسية التي يحتاجها الفريق قبل بدء التطوير.

7.2 هيكل وثيقة Mini-SRS

يجب أن ينشئ كل فريق ملفاً باسم:

docs/SRS.md

ويحتوي على الأقسام التالية:

اسم المشروع

1. وصف المشكلة

ما المشكلة الحقيقية التي يحاول المشروع حلها؟

2. الحل المقترح

كيف سيحل النظام هذه المشكلة؟

3. المستخدمون المستهدفون

من هم المستخدمون الرئيسيون للنظام؟

4. نطاق المشروع

ما الوظائف التي ستكون ضمن المشروع؟

5. خارج نطاق المشروع

ما الوظائف التي لن يتم تنفيذها في النسخة الحالية؟

6. المتطلبات الوظيفية

- FR-01:
- FR-02:
- FR-03:

7. المتطلبات غير الوظيفية

- NFR-01:
- NFR-02:
- NFR-03:

8. القيود والافتراضات

ما القيود التقنية أو الزمنية أو التنظيمية؟

حالات الحافة والمخاطر الأولية. 9. ##
ما الحالات غير الطبيعية التي يجب التفكير فيها؟

7.3 مثال Mini-SRS

اسم المشروع

نظام تنظيم مهام الطلاب.

وصف المشكلة

يعاني بعض الطلاب من تشتت الواجبات ومواعيد التسليم بين مجموعات المحادثة والمنصات المختلفة، مما يؤدي إلى نسيان بعض المهام أو تسليمها متأخرة.

الحل المقترح

تطبيق ويب بسيط يسمح للطلاب بإضافة المهام الدراسية وتنظيمها حسب المقرر والموعود النهائي وحالة الإنجاز.

المستخدمون المستهدفون

- الطالب.
- مشرف النظام.

نطاق المشروع

- تسجيل الحسابات.
- تسجيل الدخول.
- إنشاء المهام.
- تعديل المهام.
- حذف المهام.
- تحديد الموعد النهائي.
- تغيير حالة المهمة.
- تصفية المهام حسب المقرر أو الحالة.

خارج نطاق المشروع

- المحادثات الفورية.
- الاجتماعات المرئية.
- الدفع الإلكتروني.
- الربط المباشر مع أنظمة الجامعات.

متطلبات وظيفية

- **FR-01:** يجب أن يتمكن الطالب من إنشاء حساب.
- **FR-02:** يجب أن يتمكن الطالب من تسجيل الدخول.
- **FR-03:** يجب أن يتمكن الطالب من إضافة مهمة.

- **FR-04:** يجب أن يتمكن الطالب من تعديل مهامه.
- **FR-05:** يجب أن يتمكن الطالب من تعليم المهمة كمكتملة.

متطلبات غير وظيفية

- **NFR-01:** يجب حماية كلمات المرور باستخدام خوارزمية تجزئة آمنة.
- **NFR-02:** يجب أن تكون الواجهة قابلة للاستخدام على الهاتف.
- **NFR-03:** يجب ألا يتمكن المستخدم من الوصول إلى بيانات مستخدم آخر.
- **NFR-04:** يجب ألا يتجاوز وقت تحميل قائمة المهام ثانيتين في الظروف الطبيعية.

8. قصص المستخدم User Stories

8.1 ما قصة المستخدم؟

قصة المستخدم هي وصف مختصر للحاجة من وجهة نظر المستخدم المستفيد من الخاصية.

الصيغة الأساسية هي:

بصفتي [نوع المستخدم]
أريد [الوظيفة أو الهدف]
حتى أتمكن من [القيمة أو الفائدة]

باللغة الإنجليزية:

As a [user type],
I want [goal],
so that [benefit].

8.2 مثال جيد

، بصفتي طالباً ،
أريد إضافة موعد نهائي لكل مهمة
. حتى أتمكن من معرفة المهام التي يجب إنجازها أولاً

هذا المثال يوضح:

- من هو المستخدم؟
- ما الذي يريده؟
- لماذا يريده؟

8.3 مثال ضعيف

Bootstrap وLaravel إنشاء صفحة مهام باستخدام

هذه ليست قصة مستخدم جيدة؛ لأنها:

- تركز على التقنية وليس على المستخدم.
- لا توضح المشكلة.
- لا توضح القيمة.
- تفرض حلاً تقنياً قبل فهم الحاجة.

يمكن أن تكون Laravel وBootstrap جزءاً من قرار التنفيذ، لكنها ليست احتياجاً للمستخدم.

8.4 خصائص قصة المستخدم الجيدة

يُفضل أن تكون قصة المستخدم:

- مستقلة قدر الإمكان.
 - مفهومة.
 - مرتبطة بقيمة واضحة.
 - صغيرة بما يكفي لتنفيذها.
 - قابلة للنقاش.
 - قابلة للاختبار.
 - غير مرتبطة بتفاصيل تقنية غير ضرورية.
-

9. معايير القبول Acceptance Criteria

9.1 ما معايير القبول؟

معايير القبول هي الشروط التي يجب أن تتحقق حتى تُعتبر قصة المستخدم مكتملة.

بدون معايير قبول، قد يقول المطور إن المهمة انتهت، بينما يرى صاحب المشروع أنها لم تحقق المطلوب.

معايير القبول تحول العبارات العامة إلى سلوك قابل للفحص والاختبار.

9.2 صيغة Given-When-Then

يمكن كتابة معايير القبول باستخدام الصيغة التالية:

Given: الوضع الابتدائي
When: الإجراء الذي ينفذه المستخدم
Then: النتيجة المتوقعة

مثال

قصة المستخدم:

، بصفتي طالباً ،
أريد إضافة مهمة جديدة
حتى أتمكن من تنظيم واجباتي

معايير القبول:

Given أن الطالب سجل الدخول
When يدخل عنوان المهمة ويضغط على زر الحفظ
Then يتم إنشاء المهمة وتظهر في قائمة مهامه

Given أن حقل عنوان المهمة فارغ
When يضغط الطالب على زر الحفظ
Then لا يتم إنشاء المهمة وتظهر رسالة توضح أن العنوان مطلوب

Given أن الطالب أدخل موعداً نهائياً سابقاً لتاريخ اليوم
When يحاول حفظ المهمة
Then يرفض النظام الإدخال أو يطلب تأكيداً وفق قاعدة العمل المعتمدة

9.3 كيف نكتب معايير قبول جيدة؟

يجب أن تكون معايير القبول:

- واضحة.
- محددة.
- قابلة للاختبار.
- مرتبطة مباشرة بقصة المستخدم.
- خالية من الكلمات الغامضة.

كلمات غامضة يجب تجنبها

- سريع.
- سهل.

- مناسب.
- احترافي.
- يعمل جيداً.
- آمن جداً.
- يدعم عدداً كبيراً من المستخدمين.

بدلاً من:

• يجب أن تكون الصفحة سريعة

اكتب:

• يجب أن يتم تحميل الصفحة خلال ثانيتين في الظروف الطبيعية

10. حالات الحافة Edge Cases

10.1 ما حالة الحافة؟

حالة الحافة هي موقف غير شائع أو قيمة تقع عند حدود الاستخدام الطبيعي للنظام.

المستخدم عادةً يفكر في المسار الطبيعي:

1. يدخل بيانات صحيحة.
2. يضغط على الحفظ.
3. تنجح العملية.

أما مهندس البرمجيات فيفكر أيضاً في الأسئلة التالية:

- ماذا لو كان الحقل فارغاً؟
- ماذا لو تكرر الإدخال؟
- ماذا لو انقطع الإنترنت؟
- ماذا لو ضغط المستخدم على الزر مرتين؟
- ماذا لو كان التاريخ في الماضي؟
- ماذا لو أدخل المستخدم نصاً طويلاً جداً؟
- ماذا لو حاول الوصول إلى بيانات مستخدم آخر؟
- ماذا لو انتهت الجلسة أثناء العملية؟
- ماذا لو أرسل المستخدم الطلب نفسه أكثر من مرة؟
- ماذا لو لم تكن لديه صلاحية تنفيذ الإجراء؟

10.2 مثال لحالات حافة في تسجيل الحساب

- البريد الإلكتروني مستخدم سابقاً.
- البريد الإلكتروني بصيغة غير صحيحة.
- كلمة المرور قصيرة.
- حقول مطلوبة فارغة.
- الاسم يحتوي على عدد كبير جداً من الحروف.
- المستخدم ضغط زر التسجيل أكثر من مرة.
- قاعدة البيانات غير متاحة.
- تم إرسال الطلب بعد انتهاء صلاحية رمز الحماية.
- محاولة تسجيل الحساب باستخدام بيانات ضارة.
- اختلاف كلمة المرور عن تأكيد كلمة المرور.

10.3 تصنيف حالات الحافة

يمكن البحث عن حالات الحافة ضمن الفئات التالية:

البيانات

- بيانات فارغة.
- بيانات مكررة.
- بيانات طويلة جداً.
- بيانات بنوع غير صحيح.
- رموز خاصة.
- قيم سالبة أو صفرية.

المستخدم والصلاحيات

- مستخدم غير مسجل.
- مستخدم لا يملك صلاحية.
- مستخدم محظور.
- محاولة الوصول إلى بيانات مستخدم آخر.

الوقت والتاريخ

- تاريخ في الماضي.
- تاريخ نهاية يسبق تاريخ البداية.
- اختلاف المنطقة الزمنية.
- انتهاء صلاحية الرابط أو الجلسة.

الشبكة والنظام

- انقطاع الاتصال.
- بطء الاستجابة.
- فشل خدمة خارجية.
- فشل قاعدة البيانات.

- إرسال الطلب أكثر من مرة.

التزامن

- مستخدمان يعدلان السجل نفسه.
- حذف البيانات أثناء استخدامها.
- تكرار تنفيذ عملية مالية أو حساسة.

11. إدارة العمل: Kanban Board و Backlog

11.1 قائمة الأعمال Backlog

الـ Backlog هي القائمة التي تحتوي على الأعمال والخصائص والتحسينات والأخطاء التي قد يحتاج الفريق إلى تنفيذها.

يمكن أن تتضمن:

- قصص المستخدم.
- الأخطاء البرمجية.
- التحسينات.
- الأبحاث التقنية.
- المهام التوثيقية.
- أعمال الاختبار.

وجود المهمة في الـ Backlog لا يعني أن الفريق بدأ تنفيذها.

11.2 لوحة Kanban

لوحة Kanban تعرض حالة العمل الحالي بصرياً.

اللوحة المقترحة في المحاضرة تتكون من الأعمدة التالية:

Backlog → Ready → In Progress → In Review → Done

Backlog

أعمال معروفة، لكنها غير جاهزة أو غير محددة للتنفيذ حالياً.

Ready

مهام واضحة ولها معايير قبول ويمكن البدء بها.

In Progress

مهام يعمل عليها أحد أعضاء الفريق حالياً.

In Review

تم تنفيذ المهمة وفتح Pull Request لها، وهي قيد المراجعة.

Done

تمت مراجعة المهمة ودمجها وتحققت معايير القبول.

11.3 الفرق بين Backlog واللوحه

Kanban Board	Backlog
عرض مرئي لحالة سير العمل	قائمة بالأعمال المحتملة والمطلوبة
تركز على تقدم العمل	قد تحتوي على أعمال مستقبلية
تساعد في المتابعة وتقليل الفوضى	تساعد في التخطيط والأولوية
توضح هل المهمة جاهزة أو قيد التنفيذ أو المراجعة	لا تعني أن المهمة بدأت

11.4 قاعدة مهمة

لا تنقل المهمة إلى **Done** لمجرد أن المطور انتهى من كتابة الكود.

تعتبر المهمة منجزة عندما:

- تم تنفيذ معايير القبول.
- تم اختبار السلوك.
- تم رفع التغييرات.
- تم فتح Pull Request.
- تمت مراجعة التغييرات.
- تم دمجها في الفرع الرئيسي.
- تم إغلاق Issue المرتبطة بها.

12. الفرق بين Git و GitHub

Git

Git هو نظام لإدارة إصدارات الملفات وتتبع التغييرات.

يساعد الفريق على:

- معرفة من غيّر الملف.
- معرفة ما الذي تم تغييره.
- العودة إلى إصدار سابق.
- إنشاء فروع مستقلة.
- دمج أعمال أعضاء الفريق.
- تقليل خطر فقدان العمل.

GitHub

GitHub منصة تستضيف مستودعات Git وتضيف أدوات للتعاون مثل:

- .Issues
- .Pull Requests
- .Code Review
- .Project Boards
- .Discussions
- .Actions
- إدارة الصلاحيات.
- توثيق المشروع.

يمكن استخدام Git دون GitHub، لكن GitHub يجعل التعاون بين الفرق أكثر تنظيماً.

13. دورة العمل باستخدام Git و GitHub

الدورة الأساسية التي سيستخدمها الفريق هي:

```
Requirement
↓
Issue
↓
Branch
↓
Code or Documentation Changes
↓
Commit
```

↓
Push
↓
Pull Request
↓
Code Review
↓
Merge
↓
Close Issue

Issue 13.1

الـ Issue تمثل عملاً مطلوباً أو مشكلة يجب حلها.

قد تكون:

- Story: قصة مستخدم.
- Task: مهمة تقنية.
- Bug: خطأ برمجي.
- Documentation: مهمة توثيق.
- Improvement: تحسين.

يجب أن تحتوي الـ Issue الجيدة على:

- عنوان واضح.
- وصف المشكلة أو الحاجة.
- قصة المستخدم.
- معايير القبول.
- حالات الحافة.
- الأولوية.
- الشخص المسؤول.
- Labels مناسبة.

Branch 13.2

الـ Branch هو مسار عمل منفصل عن الفرع الرئيسي.

بدلاً من أن يعدل جميع الطلاب مباشرة على الفرع `main`، ينشئ كل طالب فرعاً خاصاً بالمهمة التي ينفذها.

أمثلة لتسمية الفروع

```
feature/issue-3-create-login  
fix/issue-7-login-validation  
docs/issue-2-update-srs  
test/issue-10-login-tests
```

القاعدة المستخدمة في المقرر

```
type/issue-number-short-description
```

Commit 13.3

Commit هو تسجيل لمجموعة من التغييرات مع رسالة تشرح ما تم تنفيذه.

رسالة ضعيفة

```
update
```

```
changes
```

```
final
```

رسالة أفضل

```
Add acceptance criteria for login story
```

```
Create initial mini SRS document
```

```
Fix validation for empty task title
```

القاعدة

يجب أن تصف رسالة Commit التغيير بوضوح، وليس مجرد القول إن شيئاً قد تغير.

Pull Request 13.4

الـ Pull Request هو طلب لدمج تغييرات Branch معين في Branch آخر، غالباً `main`.

يُستخدم من أجل:

- شرح ما تم تغييره.
- ربط التغيير بالـ Issue.
- مراجعة الملفات.
- اكتشاف الأخطاء.
- مناقشة القرارات.
- الموافقة قبل الدمج.

الـ Pull Request ليس مجرد زر تقني، بل هو نقطة ضبط جودة.

14. النشاط العملي الجماعي

14.1 تشكيل الفرق

يُقسم الطلاب إلى فرق، ويفضل أن يتكون كل فريق من:

- 3 إلى 5 طلاب.

يحدد الفريق الأدوار الأولية التالية:

Team Coordinator

- ينظم العمل.
- يتأكد من توزيع المهام.
- يتابع لوحة المشروع.
- لا يعني دوره أنه ينفذ كل شيء.

Requirements Owner

- يقود إعداد وثيقة SRS.
- يتأكد من وضوح قصص المستخدم.
- يراجع معايير القبول.

Repository Maintainer

- ينشئ المستودع.
- يدعو أعضاء الفريق.
- يضبط الفروع والملفات الأساسية.

Developers

- ينشئون الفروع.
- ينفذون التعديلات.
- يرفعون Pull Requests.

Reviewer

- يراجع التغييرات.
- يتأكد من ارتباطها بالIssue.
- يكتب ملاحظة أو موافقة على Pull Request.

يمكن أن يتولى الطالب أكثر من دور في الفرق الصغيرة.

14.2 اختيار فكرة المشروع

يختار الفريق فكرة مشروع واحدة.

يجب أن تكون الفكرة:

- مرتبطة بمشكلة واضحة.
- لها مستخدم محدد.
- قابلة للتنفيذ خلال الفصل.
- ليست ضخمة بصورة غير واقعية.
- تحتوي على وظائف يمكن تقسيمها إلى مهام.
- لا تعتمد بالكامل على خدمات خارجية معقدة.

أمثلة مناسبة

- نظام إدارة مهام الطلاب.
- نظام حجز مواعيد مصغر.
- نظام إدارة مكتبة.
- نظام تسجيل حضور.
- نظام متابعة مشاريع التخرج.
- نظام بلاغات صيانة.
- نظام تنظيم فعاليات جامعية.
- نظام متابعة طلبات التدريب.

أمثلة غير مناسبة بصيغتها العامة

- إنشاء بديل كامل لفيسبوك.
- إنشاء نظام مصرفي عالمي.
- إنشاء منصة مثل أمازون.
- إنشاء نظام ذكاء اصطناعي شامل.

يمكن تحويل الفكرة الكبيرة إلى نطاق صغير. على سبيل المثال:

بدلاً من:

منصة تجارة إلكترونية كاملة

يمكن اختيار:

نظام مصغر لإدارة وعرض طلبات متجر جامعي واحد

15. الخطوة الأولى: إنشاء Mini-SRS

ينشئ الفريق الملف التالي:

docs/SRS.md

ويكتب فيه:

- اسم المشروع.
- المشكلة.
- المستخدمون.
- الحل المقترح.
- النطاق.
- ما هو خارج النطاق.
- خمسة متطلبات وظيفية على الأقل.
- ثلاثة متطلبات غير وظيفية على الأقل.
- خمسة Edge Cases على الأقل.
- القيود والافتراضات.

يجب أن تكون المتطلبات مرقمة:

FR-01

FR-02

FR-03

NFR-01

NFR-02

NFR-03

16. الخطوة الثانية: تأسيس مستودع GitHub

16.1 إنشاء المستودع

ينشئ أحد أعضاء الفريق مستودعاً جديداً على GitHub.

صيغة الاسم المقترحة:

se-lab-team-01-project-name

مثال:

se-lab-team-03-student-task-manager

16.2 الملفات الأساسية

يجب أن يحتوى المستودع مبدئياً على:

README.md
.gitignore
AI_Log.md
docs/SRS.md

وقد يصبح هيكل المشروع كالتالي:

```
project-root/  
|  
├─ README.md  
├─ .gitignore  
├─ AI_Log.md  
|  
└─ docs/  
    └─ SRS.md
```

16.3 محتويات README

يجب أن يحتوى README.md على:

اسم المشروع

وصف مختصر

. وصف المشكلة والحل في فقرة قصيرة

أعضاء الفريق

- اسم الطالب الأول
- اسم الطالب الثاني
- اسم الطالب الثالث

التقنيات المتوقعة

- Laravel
- MySQL
- Git
- GitHub

وثائق المشروع

- docs/SRS.md وثيقة المتطلبات

طريقة العمل

. لإدارة العمل Pull Requests و Branches و GitHub Issues يستخدم الفريق

17. الخطوة الثالثة: إعداد GitHub Project Board

ينشئ الفريق لوحة مشروع ويربطها بالمستودع.

الأعمدة المطلوبة:

Backlog
Ready
In Progress
In Review
Done

يجب ألا تُترك جميع المهام في عمود واحد.

أثناء العمل:

1. تبدأ Issue في Backlog أو Ready.
2. عند بدء التنفيذ تنتقل إلى In Progress.
3. عند فتح Pull Request تنتقل إلى In Review.
4. بعد المراجعة والدمج تنتقل إلى Done.

18. الخطوة الرابعة: إنشاء Issues

ينشئ الفريق خمس Issues على الأقل.

كل Issue يجب أن تحتوي على:

- عنوان واضح.
- قصة مستخدم.
- وصف مختصر.
- معايير قبول.
- حالات حافة.
- Assignee.
- Label.
- أولوية إن أمكن.

18.1 قالب Issue مقترح

User Story

، بصفتي [نوع المستخدم]
، أريد [الوظيفة]
. حتى أتمكن من [الفائدة]

Description

، وصف مختصر للخاصية أو المشكلة

Acceptance Criteria

- [] المعيار الأول.
- [] المعيار الثاني.
- [] المعيار الثالث.

Edge Cases

- الحالة الأولى.
- الحالة الثانية.
- الحالة الثالثة.

Related Requirement

FR-XX

Definition of Done

- [] تم تنفيذ جميع معايير القبول
- [] تم اختبار التغيير
- [] Pull Request تم فتح
- [] تمت مراجعة التغييرات
- [] تم تحديث التوثيق عند الحاجة

18.2 مثال Issue مكتملة

العنوان

Allow students to create a new task

المحتوى

User Story

، بصفتي طالباً
، أريد إنشاء مهمة دراسية جديدة
حتى أتمكن من تنظيم الواجبات المطلوبة مني

Description

يجب توفير نموذج يسمح للطلاب بإدخال عنوان المهمة ووصفها
والمقرر والموعود النهائي

Acceptance Criteria

- [] يجب أن يكون عنوان المهمة مطلوباً
- [] يجب ربط المهمة بالمستخدم الذي أنشأها
- [] تظهر المهمة بعد الحفظ في قائمة مهام المستخدم
- [] تظهر رسالة نجاح بعد إنشاء المهمة
- [] لا يمكن للمستخدم إنشاء مهمة بعنوان فارغ

Edge Cases

- إدخال عنوان يحتوي على مسافات فقط
- إدخال عنوان أطول من الحد المسموح
- اختيار تاريخ سابق
- الضغط على زر الحفظ مرتين
- انقطاع الاتصال أثناء الإرسال

Related Requirement

FR-03

19. الخطوة الخامسة: تطبيق دورة Git

19.1 التحقق من تثبيت Git

يفتح الطالب Terminal أو Command Prompt وينفذ:

```
git --version
```

19.2 ضبط بيانات المستخدم

```
git config --global user.name "Student Name"  
git config --global user.email "student@example.com"
```

يجب استخدام بريد مرتبط بحساب GitHub عند الإمكان.

19.3 نسخ المستودع Clone

```
git clone https://github.com/USERNAME/REPOSITORY.git
```

ثم الانتقال إلى مجلد المشروع:

```
cd REPOSITORY
```

19.4 التأكد من الفرع الحالي

```
git branch
```

تحديث الفرع الرئيسي قبل إنشاء فرع جديد:

```
git checkout main  
git pull origin main
```

19.5 إنشاء Branch

مثال لمهمة رقم 3:

```
git checkout -b docs/issue-3-add-srs
```

أو باستخدام الأمر الحديث:

```
git switch -c docs/issue-3-add-srs
```

19.6 إجراء تعديل

يفتح الطالب الملف المسؤول عنه، مثل:

```
docs/SRS.md
```

ثم يضيف أو يعدل المحتوى المطلوب.

19.7 فحص الملفات المعدلة

```
git status
```

عرض الاختلافات:

```
git diff
```

19.8 إضافة الملفات إلى Staging Area

لإضافة ملف محدد:

```
git add docs/SRS.md
```

لإضافة جميع الملفات المعدلة:

```
git add .
```

يفضل عدم استخدام `git add .` دون مراجعة الملفات التي سيتم رفعها.

19.9 إنشاء Commit

```
git commit -m "Add initial mini SRS document"
```

يجب أن يمثل كل Commit تغييراً منطقياً واضحاً.

لا يُفضل جمع تعديلات كثيرة غير مرتبطة داخل Commit واحدة.

19.10 رفع الفرع Push

```
git push -u origin docs/issue-3-add-srs
```

بعد تنفيذ الأمر الأول، يمكن استخدام:

```
git push
```

20. الخطوة السادسة: فتح Pull Request

بعد رفع الفرع إلى GitHub:

1. يفتح الطالب المستودع.
2. يختار إنشاء Pull Request.
3. يحدد الفرع المصدر.
4. يتأكد أن الفرع المستهدف هو `main`.
5. يكتب عنواناً واضحاً.
6. يشرح التغييرات.
7. يربط Pull Request بالـ Issue.

20.1 قالب Pull Request

Summary

وصف مختصر للتغيير الذي تم تنفيذه.

Related Issue

Closes #3

Changes

- تمت إضافة وصف المشكلة.
- تمت إضافة المتطلبات الوظيفية.
- تمت إضافة المتطلبات غير الوظيفية.
- تمت إضافة حالات الحافة.

Testing or Verification

- [] Markdown. تمت مراجعة تنسيق
- [] تم التحقق من الروابط
- [] Issue. تم التأكد من توافق المحتوى مع الـ

Checklist

- [] اسم الفرع يتبع الصيغة المعتمدة
- [] واضحة Commit رسالة
- [] لا توجد ملفات غير مطلوبة
- [] تمت مراجعة التغييرات ذاتياً
- [] تم تحديث الوثائق عند الحاجة

استخدام:

Closes #3

يساعد GitHub على إغلاق Issue رقم 3 تلقائياً عند دمج Pull Request.

21. مراجعة Pull Request

يجب أن يراجع Pull Request طالب آخر غير الطالب الذي أنشأه.

يراجع الطالب:

- هل التغيير مرتبط بالIssue؟
- هل تم تنفيذ المطلوب فقط؟
- هل توجد أخطاء واضحة؟
- هل التسمية مفهومة؟
- هل الملف منظم؟
- هل معايير القبول مكتملة؟
- هل توجد معلومات ناقصة؟
- هل تم إدخال ملفات حساسة؟
- هل يحتاج التوثيق إلى تحديث؟

يمكن للمراجع:

- الموافقة Approve.
- كتابة تعليق.
- طلب تعديل Request Changes.

21.1 مثال لتعليق مراجعة جيد

FR-04 تمت صياغة المتطلبات الوظيفية بوضوح، لكن المتطلب لا يحدد من يملك صلاحية تعديل المهمة. أقترح توضيح أن الطالب يستطيع تعديل المهام التي أنشأها فقط.

21.2 مثال لتعليق غير مفيد

.جيد

المراجعة الاحترافية يجب أن تقدم ملاحظة محددة وقابلة للتنفيذ.

22. قواعد الدمج Merge

لا يتم الدمج في `main` إلا بعد:

- اكتمال المطلوب.
- وجود Pull Request.
- مراجعة طالب آخر.
- معالجة الملاحظات.
- التأكد من عدم وجود ملفات حساسة.
- التأكد من ارتباط Pull Request بالIssue.

يمنع في النشاط العملي تعديل الفرع `main` مباشرة، إلا في التهيئة الأولى للمستودع عند الضرورة.

23. التعامل مع تعارضات الدمج

يحدث Merge Conflict عندما يعدل أكثر من شخص المنطقة نفسها من الملف، ولا يستطيع Git تحديد النسخة الصحيحة تلقائياً.

عند حدوث تعارض:

1. لا تحذف الملف.
2. لا تنشئ مستودعاً جديداً.
3. اقرأ مناطق التعارض.
4. ناقش النسخة الصحيحة مع صاحب التعديل الآخر.
5. عدل الملف يدوياً.
6. احذف علامات التعارض.
7. اختبر النتيجة.
8. أنشئ Commit جديداً لحل التعارض.

علامات التعارض تظهر عادة بالشكل التالي:

```
<<<<<<< HEAD
المحتوى الموجود في الفرع الحالي
=====
المحتوى القادم من الفرع الآخر
>>>>>>> branch-name
```

يجب اختيار المحتوى الصحيح أو دمج النسختين، ثم إزالة جميع العلامات.

24. استخدام الذكاء الاصطناعي

يسمح باستخدام أدوات الذكاء الاصطناعي في هذه المحاضرة باعتبارها أدوات مساعدة، وليس بديلاً عن التحليل الهندسي.

24.1 الاستخدامات المسموحة

يمكن استخدام الذكاء الاصطناعي من أجل:

- اقتراح حالات حافة.
- تحسين صياغة قصة المستخدم.
- مراجعة وضوح معايير القبول.
- اقتراح أسئلة يجب طرحها حول المتطلبات.
- اكتشاف حالات غامضة أو ناقصة.
- تحسين تنسيق Markdown.

• شرح أمر Git لم يفهمه الطالب.

24.2 الاستخدامات غير المقبولة

لا يقبل من الفريق:

- نسخ إجابة كاملة دون فهمها.
- إنشاء متطلبات لا يعرف الفريق معناها.
- إضافة Edge Cases دون مراجعتها.
- تسليم محتوى مولد دون توثيق.
- استخدام الذكاء الاصطناعي لإخفاء عدم مشاركة أعضاء الفريق.
- قبول اقتراحات الأداة باعتبارها صحيحة تلقائياً.
- رفع بيانات حساسة أو كلمات مرور إلى أدوات خارجية.

الطالب مسؤول عن صحة ما يقدمه، سواء كتبه بنفسه أو اقترحه أداة ذكاء اصطناعي.

25. ملف AI_Log.md

يجب إنشاء ملف في جذر المشروع باسم:

AI_Log.md

يسجل فيه الفريق كل استخدام مؤثر للذكاء الاصطناعي.

25.1 القالب المطلوب

AI Usage Log

Entry 01

Date

2026-XX-XX

Student

اسم الطالب

Tool

اسم الأداة المستخدمة

Purpose

.استخراج حالات حافة لقصة تسجيل الدخول

Prompt Summary

.طلبنا من الأداة اقتراح حالات غير طبيعية قد تحدث أثناء تسجيل الدخول

AI Suggestions

- حساب غير مفعل.
- عدد كبير من محاولات الدخول الفاشلة.
- انتهاء الجلسة.
- إدخال بريد بصيغة غير صحيحة.

Accepted Suggestions

- عدد كبير من محاولات الدخول الفاشلة.
- حساب غير مفعل.

Rejected Suggestions

- تسجيل الدخول باستخدام البصمة.

Reason for Rejection

.التحقق بالبصمة خارج نطاق النسخة الحالية من المشروع

Human Review

.راجع الفريق الاقتراحات وربط الحالات المقبولة بمتطلبات المشروع

25.2 لماذا نوثق استخدام الذكاء الاصطناعي؟

الهدف ليس منع استخدام الذكاء الاصطناعي، بل تدريب الطالب على:

- الشفافية.
- التفكير النقدي.
- مراجعة المخرجات.
- التمييز بين الاقتراح والقرار.
- تحمل المسؤولية الهندسية.
- توثيق الأدوات المستخدمة في العمل.

في العمل الاحترافي، لا تكمن المشكلة في استخدام الأداة، بل في استخدام مخرجاتها دون فهم أو مراجعة أو مساءلة.

26. المخرجات المطلوبة Deliverables

في نهاية المحاضرة، يجب أن يقدم كل فريق المخرجات التالية:

26.1 مستودع GitHub

- اسم مستودع واضح.
- إضافة جميع أعضاء الفريق.
- وجود README.md.
- وجود .gitignore.
- وجود AI_Log.md.
- وجود مجلد docs.

26.2 وثيقة Mini-SRS

- وصف المشكلة.
- الحل المقترح.
- المستخدمون المستهدفون.
- نطاق المشروع.
- ما هو خارج النطاق.
- خمسة متطلبات وظيفية على الأقل.
- ثلاثة متطلبات غير وظيفية على الأقل.
- خمسة Edge Cases على الأقل.
- قيود وافتراضات المشروع.

26.3 إدارة المهام

- إنشاء GitHub Project.
- وجود أعمدة العمل المطلوبة.
- إنشاء خمس Issues على الأقل.
- كتابة كل Issue بصيغة قصة مستخدم.
- إضافة Acceptance Criteria.
- إضافة Edge Cases.
- تعيين مسؤول لكل Issue.

26.4 تطبيق Git Workflow

- إنشاء Branch واحد على الأقل.
- إجراء تعديل حقيقي.
- إنشاء Commit واضح.
- تنفيذ Push.
- فتح Pull Request.
- ربط Pull Request بالIssue.
- مراجعة Pull Request من طالب آخر.
- دمج التغيير في main.

26.5 توثيق الذكاء الاصطناعي

- وجود AI_Log.md.
- توثيق الأداة المستخدمة.

- توثيق الغرض من الاستخدام.
- توضيح الاقتراحات المقبولة والمرفوضة.
- إثبات وجود مراجعة بشرية.

27. آلية التقييم

الدرجة	العنصر
10	وضوح فكرة المشروع والمشكلة
20	جودة وثيقة Mini-SRS
10	التمييز بين المتطلبات الوظيفية وغير الوظيفية
15	جودة User Stories و Acceptance Criteria
10	اكتشاف Edge Cases واقعية
10	تنظيم Issues و Project Board
10	محة تطبيق Push و Commit و Branch
10	جودة Pull Request والمراجعة
5	توثيق استخدام الذكاء الاصطناعي
100	المجموع

28. معايير جودة العمل

ممتاز

- المشكلة واضحة ومحددة.
- المتطلبات قابلة للاختبار.
- قصص المستخدم مرتبطة بقيمة حقيقية.
- حالات الحافة متنوعة وواقعية.
- Issues منظمة.
- أسماء الفروع والCommits واضحة.
- Pull Request موثقة ومراجعة.
- مساهمة أعضاء الفريق ظاهرة.
- استخدام الذكاء الاصطناعي موثق ومراجع.

مقبول

- الفكرة مفهومة لكنها عامة.

- بعض المتطلبات غامضة.
- معايير القبول محدودة.
- توجد Issues، لكن تنظيمها غير مكتمل.
- تم تطبيق Git Workflow مع بعض الأخطاء البسيطة.

غير مقبول

- البدء بالكود دون متطلبات.
- نسخ محتوى عام لا يرتبط بالمشروع.
- غياب معايير القبول.
- عدم وجود Issues حقيقية.
- العمل مباشرة على `main`.
- استخدام Commit Messages مثل `update` و `final`.
- عدم وجود Pull Request.
- تنفيذ جميع الأعمال بحساب طالب واحد.
- استخدام الذكاء الاصطناعي دون توثيق.

29. الأخطاء الشائعة

الخطأ الأول: اختيار فكرة ضخمة

بدلاً من بناء عشرات الوظائف، حدد مشكلة صغيرة وابن حلاً واضحاً لها.

الخطأ الثاني: كتابة متطلبات غامضة

. يجب أن يكون النظام سريعاً وآمناً

هذه عبارة غير قابلة للقياس.

الخطأ الثالث: كتابة التقنية بدلاً من الحاجة

. لإنشاء صفحة تسجيل دخول Laravel استخدام

هذه مهمة تقنية، وليست قصة مستخدم.

الخطأ الرابع: عدم تحديد الفائدة

. بصفتي مستخدماً، أريد زر إضافة

إضافة ماذا؟ ولماذا؟

الخطأ الخامس: العمل مباشرة على main

هذا يلغي فائدة المراجعة ويزيد خطر إدخال تغييرات غير مستقرة.

الخطأ السادس: Pull Request ضخمة

كلما كانت Pull Request صغيرة ومحددة، أصبحت مراجعتها أسهل وأكثر دقة.

الخطأ السابع: مراجعة شكلية

ضغط Approve دون قراءة التغييرات ليس مراجعة هندسية.

الخطأ الثامن: نسخ مخرجات الذكاء الاصطناعي

الأداة قد تقترح خصائص خارج نطاق المشروع أو متطلبات غير واقعية. الفريق مسؤول عن الفحص والاختيار.

30. قائمة التحقق النهائية للفريق

قبل انتهاء المحاضرة، يجب الفريق عن الأسئلة التالية:

المتطلبات

- [] هل المشكلة واضحة؟
- [] هل المستخدم المستهدف محدد؟
- [] هل كتبنا خمسة متطلبات وظيفية على الأقل؟
- [] هل كتبنا ثلاثة متطلبات غير وظيفية على الأقل؟
- [] هل يمكن اختبار كل متطلب؟
- [] هل حددنا ما هو خارج نطاق المشروع؟

User Stories

- [] هل تحدد كل قصة نوع المستخدم؟
- [] هل توضح الوظيفة المطلوبة؟
- [] هل توضح القيمة؟
- [] هل لها معايير قبول؟
- [] هل لها حالات حافة؟

GitHub

- [] هل جميع أعضاء الفريق مضافون؟
- [] هل المستودع يحتوي على الملفات المطلوبة؟
- [] هل أنشأنا Project Board؟
- [] هل أنشأنا خمس Issues على الأقل؟

- [] هل لكل Issue مسؤول؟

Git

- [] هل أنشأنا Branch مستقلاً؟
- [] هل اسم الفرع واضح؟
- [] هل راجعنا التغييرات باستخدام `git diff`؟
- [] هل رسالة Commit واضحة؟
- [] هل رفعنا الفرع إلى GitHub؟
- [] هل فتحنا Pull Request؟
- [] هل راجع Pull Request عضو آخر؟
- [] هل تم الدمج في `main`؟

الذكاء الاصطناعي

- [] هل وثقنا الأداة المستخدمة؟
- [] هل سجلنا الغرض من الاستخدام؟
- [] هل راجعنا الاقتراحات؟
- [] هل وضعنا ما تم قبوله ورفضه؟
- [] هل الملف `AI_Log.md` موجود؟

31. الخلاصة

هندسة البرمجيات لا تبدأ من لوحة المفاتيح، بل تبدأ من فهم المشكلة.

المشروع الاحترافي لا يُقاس بعدد الأسطر المكتوبة، بل بقدرة الفريق على:

- فهم احتياجات المستخدم.
- تحويل الاحتياجات إلى متطلبات واضحة.
- تقسيم العمل إلى مهام صغيرة.
- العمل دون إفساد تغييرات الآخرين.
- مراجعة القرارات والتغييرات.
- توثيق ما تم تنفيذه.
- بناء نظام يمكن تطويره وصيانته مستقبلاً.

خلال هذه المحاضرة، لا يُطلب من الفريق بناء ناطحة السحاب كاملة، بل يُطلب منه وضع أساس هندسي صحيح قبل البدء بالبناء.

المعادلة التي سنستخدمها طوال المقرر هي:

مشكلة واضحة
+ متطلبات قابلة للاختيار
+ مهام منظمة
+ `Git Workflow` منضبط

+ مراجعة جماعية

= مشروع برمجي قابل للنجاح والتوسع